

Tabella del Programma di Lavoro

Fase	Settimana	Attività	Concetti Python	Output
1	Sett. 1	Fondamenta teoriche e setup Ripasso del paper. Installazione di Python (Anaconda), Jupyter Notebook, librerie <code>numpy</code> , <code>matplotlib</code> , <code>pandas</code> . Primo script: generare una distribuzione di Pareto e visualizzarla.	<code>pip/conda</code> , ambienti virtuali, <code>import</code> di librerie, <code>numpy.random</code> , <code>matplotlib.pyplot</code>	Script di distribuzione Pareto
2	Sett. 2	Modellare i trader Creare una classe <code>Trader</code> con attributi <i>cash</i> e <i>bitcoin</i> . Implementare le sottoclassi <code>RandomTrader</code> e <code>Chartist</code> . Testare la generazione di una popolazione con ricchezza distribuita secondo Zipf.	Classi e oggetti (<code>class</code> , <code>__init__</code> , ereditarietà), liste di oggetti	Classe <code>Trader</code> funzionante
3	Sett. 3	Ordini e Order Book Creare le classi <code>BuyOrder</code> e <code>SellOrder</code> . Implementare la classe <code>OrderBook</code> con liste ordinate di ordini. Aggiungere il metodo <code>match()</code> per incrociare ordini compatibili e calcolare il prezzo di transazione.	Classi, liste ordinate (<code>sorted</code> , <code>bisect</code>), metodi, condizionali	Order Book con matching
4	Sett. 4	Ciclo di simulazione Implementare il loop principale: ad ogni <i>time step</i> attivare una frazione casuale di trader, generare i loro ordini, inserirli nell'order book, eseguire il matching e registrare il prezzo. Simulare 200 giorni.	<code>for</code> loop, <code>random.random()</code> , liste per serie temporali	Simulazione 200 giorni

Fase	Settimana	Attività	Concetti Python	Output
5	Sett. 5	Analisi statistica dei risultati Calcolare i rendimenti giornalieri (<i>log-returns</i>). Plottare: (a) serie temporale del prezzo, (b) istogramma dei rendimenti con curva normale sovrapposta, (c) autocorrelazione dei rendimenti assoluti. Verificare visualmente i <i>fat tail</i> e il clustering della volatilità.	numpy (log, std, correlazione), matplotlib (subplots, hist, acorr), scipy.stats	Grafici stilized facts
6	Sett. 6	Estensioni e confronto col paper Aggiungere l'ingresso di nuovi trader nel tempo (proporzionale agli indirizzi Blockchain). Aggiungere il mining (aumento graduale dei Bitcoin). Confrontare i grafici simulati con quelli del paper. Preparare la presentazione finale.	Refactoring del codice, parametri configurabili, pandas per i dati, presentazione con Jupyter	Presentazione finale

Struttura consigliata del codice

File	Contenuto
trader.py	Classi Trader, RandomTrader, Chartist
orders.py	Classi BuyOrder, SellOrder
orderbook.py	Classe OrderBook con metodi di matching
market.py	Classe Market: loop di simulazione, mining, nuovi trader
analysis.py	Funzioni per calcolo rendimenti, plot, statistiche
main.py	Script principale: parametri, avvio simulazione, output

Librerie Python necessarie

Libreria	Utilizzo nel progetto
<code>numpy</code>	Generazione numeri casuali, calcoli vettoriali, log-returns
<code>matplotlib</code>	Grafici del prezzo, istogrammi, autocorrelazione
<code>pandas</code>	Gestione serie temporali, confronto con dati reali
<code>scipy.stats</code>	Test statistici (Dickey-Fuller con <code>statsmodels</code>), kurtosi
<code>statsmodels</code>	Test Augmented Dickey-Fuller per la radice unitaria
<code>collections</code>	<code>deque</code> per la gestione efficiente dell'order book

Suggerimento per iniziare: nella prima settimana, prima di scrivere codice, disegnare su carta lo schema delle classi e delle loro relazioni. Un buon design iniziale fa risparmiare molto tempo nelle settimane successive.